TundraXR 按钮与 Brain 接口手册

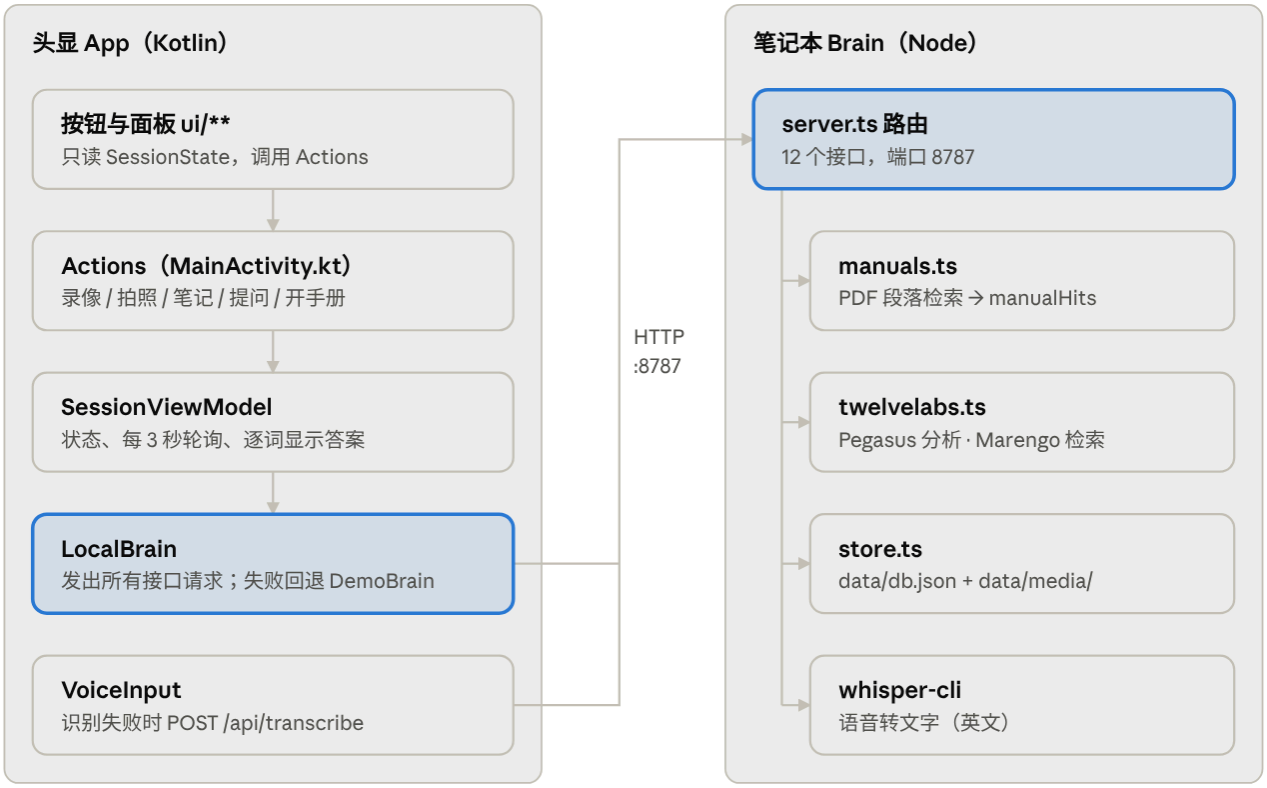
Oct 17, 2026 · @Lulu

1. 总览

头显 App 通过 HTTP 连接笔记本上的 Tundra brain（默认 `http://localhost:8787`），所有请求都集中在 `LocalBrain.kt`；只有语音兜底转写在 `VoiceInput.kt` 里单独请求。

- **连接方式：** USB 连接时，brain 每 4 秒自动执行一次 `adb reverse tcp:8787 tcp:8787`，头显上的 localhost 就是笔记本。走 Wi-Fi 时用 `./gradlew installDebug -Ptundra.serverUrl=http://<笔记本IP>:8787` 改地址。
- **身份识别：** 头显的每个请求都带 `user-agent: TundraXR/1.0`，brain 据此记录 `headsetSeenAt`，dashboard 显示“头显已连接”。
- **离线回退：** 拉不到设备列表或提问失败时，App 自动改用内置的 `DemoBrain` 示例数据；上传采集失败则标记为“Not sent · laptop offline”，不会自动重传。
- **代码分层：** 按钮（`ui/**`）→ `Actions`（`MainActivity.kt`）→ `SessionViewModel` → `TundraBrain` 接口（`LocalBrain` / `DemoBrain`）→ HTTP → brain（`server/src/server.ts`）。

头显只经 LocalBrain 与笔记本 brain 通信



系统结构 · 头显 5 层，brain 5 个模块

按钮不直接联网：请求从上往下传到 LocalBrain，再经 HTTP 到 server.ts；笔记本浏览器里的 dashboard 也连同一个 server.ts。

超时设置

请求	连接超时	读取超时
`GET` 类请求（设备列表、状态轮询）	3 秒	5 秒
`POST /api/captures`（上传媒体）	3 秒	120 秒
`POST /api/ask`	3 秒	30 秒
`POST /api/transcribe`	3 秒	60 秒

2. 按钮 → API 对照表

会联网的只有 6 类操作：录像、拍照、语音笔记、提问（文字/语音/建议）、打开手册、启动时加载。其余按钮只改本地界面状态。

2.1 会调用 brain 的按钮

按钮（位置）	Action → ViewModel	调用的 API	发出的输入	收到的输出	界面变化
Record / 停止（控制栏）	`record` → `onRecordingStarted()`；再按 → `onRecordingStopped(file)`	停止后 `POST /api/captures`	MP4 文件（FHD，含音频）+ meta `kind:"video"`、`durationSec`	新建的 Capture（`id`、`tl`）	时间线加一条视频，Sending → Analyzing → Analyzed
Take photo 相机图标（控制栏）	`photo` → `onShutter()` + `onPhotoTaken(file)`	`POST /api/captures`	JPEG 文件 + meta `kind:"photo"`	新建的 Capture	闪光效果，时间线加一张照片
Voice note 麦克风（控制栏）	`note` → `voice.toggle(Note)` → `onFinalTranscript()`	识别失败时 `POST /api/transcribe`；之后 `POST /api/captures`	转写：WAV；上传：空 body + meta `kind:"note"`、`text`	`{text}`；然后新建的 Capture	实时显示识别文字，结束后时间线加一条笔记
发送 ↑（Ask 输入框，或键盘回车）	`send` → `ask(askDraft)`	`POST /api/ask`	`{sessionId, question, device}`	`{answer, manualHits, moments}`	对话里逐词显示答案和来源标签；若引用手册则自动打开到该页
建议问题标签（Ask 面板，按设备不同）	`ask(text)`	`POST /api/ask`	同上，`question` = 标签文字	同上	同上

按钮（位置）	Action → ViewModel	调用的 API	发出的输入	收到的输出	界面变化
Ask by voice 麦克风（Ask 输入框旁）	`askVoice` → `voice.toggle(Ask)` → `ask(text)`	必要时 `POST /api/transcribe`；然后 `POST /api/ask`	WAV；然后同上	`{text}`；然后同上	输入框显示“Listening…”和实时文字
手册来源标签（答案下方，如“p.142 · Alarms”）	`openSource(source)` → `openManual(target)`	若手册还未下载：`GET /manuals/{id}.pdf`	—	PDF 文件（下载一次后缓存）	手册面板滚动到该页并高亮 `box` 区域
Manual 书本图标（控制栏）	`toggleManual`	首次显示时 `GET /manuals/{id}.pdf`	—	PDF 文件	显示/隐藏手册面板
设备名标签（Capture 面板顶部）	`selectDevice(device)`	不发新请求（用已缓存的设备列表），手册面板会重新下载对应 PDF	—	—	切换设备，清空对话，换手册和建议问题

2.2 只改本地状态的按钮

按钮	Action	作用
Ask Tundra ✦（控制栏）	`toggleAsk`	显示/隐藏 Ask 面板；窄窗口下切换到 Ask 页
Close ×（Ask / 手册面板）	`toggleAsk` / `toggleManual`	关闭面板
Zoom in / Zoom out / 双击（手册面板）	面板内部状态	缩放 PDF，不联网
输入框打字	`draft(text)`	更新 `askDraft`
Switch to Home Space / Full Space	XR `session.scene`	切换空间模式，不联网

2.3 不需要点按钮的自动请求

时机	API	用途
App 启动	`GET /api/devices`	拉取设备和手册列表；等 1.2 秒后把第一台设备当作“识别结果”
每 3 秒	`GET /api/captures?sessionId=…`	心跳（判断笔记本是否在线）+ 刷新每条采集的分析状态

3. Brain 接口详解

Brain 共 12 个接口：头显用 6 个，笔记本 dashboard 用其余 6 个。JSON 响应都带 `access-control-allow-origin: *`；出错时统一返回 `{ "error": "…" }`，状态码 400 / 404 / 500 / 501。

3.1 GET /api/devices — 设备与手册目录

输入： 无。

输出： 数组，每台设备一条。数据来自 `server/manuals/catalog.json`，没有 PDF 的设备不返回。

```
[{
  "id": "pb980", "name": "Puritan Bennett 980", "manufacturer": "Medtronic",
  "category": "Ventilator", "assetTag": "ICU-VNT-0412", "serial":
"35B1800412",
  "location": "ICU 3 · Bed 7", "lastPm": "14 Jul 2026", "openWorkOrders": 1,
  "manual": { "id": "pb980-om", "title": "PB980 Operator's Manual",
"docNumber": "PT00169219",
              "kind": "Operator's manual", "file": "pb980-operators-
manual.pdf",
              "pages": 412, "url": "/manuals/pb980-om.pdf" }
}]
```

字段	含义
`id`	设备在 brain 里的唯一编号；`/api/ask` 用它找对应手册
`assetTag` / `serial` / `location`	医院资产编号、序列号、所在位置（目前是演示值，将来来自 CMMS）
`lastPm` / `openWorkOrders`	上次预防性维护日期、未关闭工单数
`manual.docNumber`	OEM 文档号，App 里显示为手册版本
`manual.pages`	PDF 总页数
`manual.url`	PDF 下载路径；App 拼成 `baseUrl + url`

3.2 GET /manuals/{manualId}.pdf — 手册文件

输入： 路径里的手册 id；可带 `Range` 头分段下载。**输出：** `application/pdf` 文件。App 下载一次后缓存在本地，用 `PdfRenderer` 渲染。

3.3 POST /api/captures — 上传一条采集

输入： 元数据放在请求头，body 是原始媒体字节（语音笔记时为空）。这样大视频可以直接流式上传，不用 multipart。

请求头	值
`content-type`	`image/jpeg`（照片）或 `video/mp4`（视频）；笔记不带
`x-tundra-meta`	URI 编码后的 JSON，见下

```
{
  "sessionId": "s_1760700000000",
  "kind": "photo",
  "at": "02:14",
  "durationSec": 0,
  "text": "仅语音笔记有：转写文字",
  "device": { "id": "pb980", "name": "…", "manufacturer": "…", "category":
"…",
              "assetTag": "…", "serial": "…", "location": "…" }
}
```

meta 字段	含义
`sessionId`	本次会话编号，App 启动时生成（`s_` + 毫秒时间戳）；用来把采集归到同一次工作
`kind`	`photo` / `video` / `note`
`at`	距会话开始的时间，`mm:ss`
`durationSec`	视频时长（秒），其他类型为 0
`text`	语音笔记的转写文字
`device`	当前设备的身份信息，brain 原样存下

输出： `201`，完整的 Capture 记录。App 只用 `id`、`tl` 和 `analysis.findings`。

```
{
  "id": "a1b2c3d4", "sessionId": "…", "kind": "video", "at": "02:14",
  "createdAt": "2026-10-17T09:30:00.000Z", "device": { "…": "…" },
  "file": "a1b2c3d4.mp4", "mime": "video/mp4", "bytes": 18300000,
"durationSec": 12,
  "tl": { "upload": "pending", "analysis": "pending", "index": "pending" },
  "qa": []
}
```

Capture 字段	含义
`id`	brain 分配的 8 位编号；App 存为 `remoteId`，轮询时用它对应
`file` / `mime` / `bytes`	存在 `server/data/media/` 的文件名、类型、大小
`tl.upload`	视频上传到 TwelveLabs 的进度
`tl.analysis`	AI 分析进度（照片和视频）
`tl.index`	视频建索引（用于以后搜索）的进度
`tl.error`	失败原因，最多 400 字
`analysis`	分析完成后的结构化维修记录，见第 5 节
`qa`	在 dashboard 上对这段视频的提问记录

`tl` 里每一项的取值：`pending`（排队）、`running`（进行中）、`done`、`failed`、`skipped`（不做：没配 TwelveLabs key，或笔记这类没有媒体的采集）。

3.4 GET /api/captures?sessionId=… — 状态轮询

输入： 查询参数 `sessionId`（不带则返回全部）。**输出：** 该会话的 Capture 数组，结构同 3.3。App 每 3 秒调一次：成功即表示笔记本在线，并按 `id` 更新每条采集的状态（映射见第 4 节）。

3.5 POST /api/ask — 提问

输入：

```
{ "sessionId": "s_1760700000000", "question": "How do I run SST?",
  "device": { "id": "pb980", "…": "…" } }
```

输出：

```
{
  "answer": "PB980 Operator's Manual, p. 142 · Short Self Test:\n“…”\n\nAlso
see p. 143, p. 150.",
  "manualHits": [
    { "manualId": "pb980-om", "title": "PB980 Operator's Manual", "page":
142,
      "section": "Short Self Test", "text": "…", "box": [0.08, 0.31, 0.92,
0.47] }
  ],
  "moments": [ { "captureId": "a1b2c3d4", "start": 12.5, "end": 18.0, "rank":
1, "thumbnailUrl": "…" } ]
}
```

字段	含义
`answer`	给工程师看的答案：引用最匹配的手册段落原文（最多约 360 字符），再列出其他匹配页
`manualHits[].page`	PDF 里的页码（从 1 开始）
`manualHits[].section`	该段所属章节标题
`manualHits[].text`	该段的摘录
`manualHits[].box`	该段在页面上的位置 `[左，上，右，下]`，均为 0–1 的页面比例，左上角为原点；App 用它画高亮
`moments[]`	同一台设备录过的视频里与问题匹配的片段；`start` / `end` 为秒，在笔记本 dashboard 上观看

App 把每个 `manualHit` 变成一个可点击的“p.142 · Short Self Test”标签，把每个 `moment` 变成“00:12–00:18 · on laptop”标签。问题和答案也会作为一条 `kind:"question"` 的 Capture 存进会话记录。

3.6 POST /api/transcribe — 语音转文字

输入： body 为 16 kHz、单声道、16 位的 WAV，`content-type: audio/wav`。头显录音最长 20 秒，说完后静音 1.6 秒自动停止。**输出：** `{ "text": "识别出的文字" }`。没有安装 whisper 模型时返回 `501`。

3.7 仅 dashboard 使用的接口

方法 路径	输入	输出
`GET /`	—	dashboard 页面

方法 路径	输入	输出
`GET /api/state`	—	`{ twelveLabs, indexId, headsetSeenAt, addresses, captures[] }`：TwelveLabs 是否开启、索引编号、头显最后一次请求时间、笔记本 IP、全部采集
`GET /api/events`	—	SSE 事件流：`status`（每 5 秒）和 `capture`（有采集新增或状态变化时，内容为整条 Capture）
`GET /media/{file}`	支持 `Range`	照片或视频文件
`POST /api/search`	`{ query }`	`{ moments[], photos[] }`，每项带 `match`（是否真的匹配）和 `description`（一句话说明）；未配 key 返回 400
`POST /api/captures/{id}/ask`	`{ question }`	更新后的 Capture，新问答追加在 `qa` 里

4. 状态与错误处理

每条采集在头显上只显示 5 种状态之一，由 brain 返回的 `tl.analysis` 决定（`LocalBrain.toStatus()`）。

brain 返回	头显状态 `SyncState`	时间线上的小字
请求还没返回	`Sending`	—
`tl.analysis` = `skipped`	`InBrain`（已存入，不再变化）	—
`tl.analysis` = `pending` / `running`	`Analyzing`	`tl.upload` = `running` 时显示 “Uploading”
`tl.analysis` = `done`	`Analyzed`	“N findings”（`analysis.findings` 的条数）或 “No findings”
`tl.analysis` = `failed`	`Failed`	“Analysis failed”
上传请求本身失败	`Failed`	“Not sent · laptop offline”

在线判断

- `brainOnline = null`：启动后还没轮询过。
- `brainOnline = true`：最近一次 `GET /api/captures` 成功。

- `brainOnline = false`：轮询失败，或有一次上传失败。

各类失败时 App 怎么做

失败的请求	App 的处理
`GET /api/devices`	用 DemoBrain 的示例设备（Evita V500）和内置手册页
`POST /api/captures`	该条标为 Failed，不自动重传
`POST /api/ask`	改用 DemoBrain 的脚本答案
`POST /api/transcribe`	显示语音错误提示 4 秒，不生成笔记或问题
`GET /manuals/{id}.pdf`	手册面板保持“Opening manual…”

系统语音识别器有一次严重失败后，本次运行里的语音都直接走 `/api/transcribe`。

5. Brain 内部处理

Brain 收到采集后立刻存盘并返回，AI 处理在后台进行；结果写回同一条 Capture，头显通过轮询、dashboard 通过 SSE 看到更新。没有配 `TWELVELABS_API_KEY` 时，只存储不分析；之后配上 key 重启，以前的照片和视频会自动补分析。

5.1 存储

- 记录：`server/data/db.json`（写入合并，200 毫秒内最多写一次）。
- 媒体文件：`server/data/media/{id}.jpg|mp4`。
- 手册索引：`server/data/manual-index/`，PDF 更新后自动重建。

5.2 视频处理流程（TwelveLabs）

1. 上传：`assets.create` 把 MP4 传到 TwelveLabs，等待就绪（`tl.upload`）。
2. 分析：用 Pegasus 1.6 模型按固定 JSON 结构生成维修记录（`tl.analysis`）。
3. 建索引：加入 `tundra-clinical-engineering` 索引（Marengo 3.0，画面 + 声音），以后提问和搜索可以找到其中的片段（`tl.index`，最长等 15 分钟）。

照片只做第 2 步（不上传、不建索引）。语音笔记不做 AI 处理，但会作为同一会话的上下文一起发给分析模型。

5.3 分析结果 `analysis` 的字段

字段	视频	照片	含义
`summary`	是	是	2–3 句摘要（照片为 1 句）
`findings[]`	是	是	每条发现：`title`、`severity`（info / minor / major / critical）、`detail`，视频另有 `start_time` / `end_time`（秒）。头显只显示条数
`safety_concerns[]`	是	是	影响病人或员工安全的问题
`work_type`	是	—	工作类型：预防性维护、维修、巡检、校准、入库检查、排障、其他
`device_identification`	是	—	画面里可见的型号、序列号、资产标签、校准贴纸
`steps_performed[]`	是	—	技术员的操作步骤及开始时间
`parts_and_consumables[]` / `tools_and_test_equipment[]`	是	—	用到的零件耗材、工具和测试设备
`recommended_follow_up[]`	是	—	建议的后续操作
`device_status_after`	是	—	in_service / out_of_service / needs_follow_up / unknown
`documentation_gaps[]`	是	—	审核方会要求但画面里没有的记录（如未见电气安全测试结果）
`visible_labels[]`	—	是	照片里每个可读标签的 `kind` 和 `text`
`tags[]`	—	是	3–6 个搜索关键词

5.4 手册检索（回答 /api/ask）

用 `pdfjs-dist` 把每本 PDF 切成段落，记录页码、章节和位置框。提问时在当前设备的手册里取得分最高的 3 段；若开了 TwelveLabs，再搜 6 个视频片段，只保留同一台设备的前 3 个。目前答案是手册原文引用，不是大模型生成的。

5.5 语音转写

调用本机 `whisper-cli`，模型 `server/models/ggml-base.en.bin`，仅英文。启动时先用 1 秒静音预热，避免第一次提问等待约 15 秒。